

Skill - 21:

```
sahasrasagi@Sahasra: ~/OS_L  ×  +  ∨
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>
#include <time.h>

#define MAX_COMMAND 200
#define MAX_ARGS 50

void log_error(const char *message)
{
    FILE *file = fopen("shell_errors.log", "a");

    if (file == NULL)
        return;

    time_t now = time(NULL);

    fprintf(file, "[%s] %s\n", ctime(&now), message);

    fclose(file);
}

void execute_command(char *input)
{
    char *args[MAX_ARGS];
    int count = 0;

    char temp[MAX_COMMAND];
    strcpy(temp, input);

    char *token = strtok(temp, " ");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    if (count == 0)
        return;

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        log_error("fork failed");
        return;
    }

    if (pid == 0)
    {
        execvp(args[0], args);

        perror("execvp");
        exit(127);
    }

    int status;

    if (waitpid(pid, &status, 0) < 0)
    {
        perror("waitpid");
    }
}
```

```

    if (waitpid(pid, &status, 0) < 0)
    {
        perror("waitpid");
        log_error("waitpid failed");
        return;
    }

    if (WIFEXITED(status))
    {
        int code = WEXITSTATUS(status);

        if (code != 0)
        {
            printf("Command failed with exit code %d\n", code);
            log_error("Command execution failed");
        }
    }
    else if (WIFSIGNALED(status))
    {
        printf("Command terminated by signal %d\n", WTERMSIG(status));
        log_error("Command terminated by signal");
    }
}

int main()
{
    char input[MAX_COMMAND];

    printf("==== Skill 21 =====\n");

    while (1)
    {
        printf("\nmyshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "help") == 0)
        {
            printf("Commands: help, exit, pwd, ls, date, invalid commands\n");
            continue;
        }

        execute_command(input);
    }

    return 0;
}

```

```
sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_21.c -o skill_21
```

```
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_21
```

```
==== Skill 21 =====
```

```
myshell> ls
```

```

a.out      program1.c  shellforge  skill_10.c  skill_12    skill_13.c  skill_15    skill_
output.txt program2    shellforge.c skill_11    skill_12.c  skill_14    skill_15.c  skill_
program1   program2.c skill_10     skill_11.c  skill_13    skill_14.c  skill_16    skill_

```

```
myshell> hello123
```

```
execvp: No such file or directory
```

```
Command failed with exit code 127
```

```
myshell> help
```

```
Commands: help, exit, pwd, ls, date, invalid commands
```

```
myshell> exit
```

```
sahasrasagi@Sahasra:~/OS_Lab/skill$ |
```

Skill - 22:

sahasrasagi@Sahasra: ~/OS_L × + ▾

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_SIZE 100

void memory_test()
{
    char *buffer = malloc(MAX_SIZE);

    if (buffer == NULL)
    {
        perror("malloc");
        return;
    }

    strcpy(buffer, "OSSP Skill 22 Memory Test");

    printf("Allocated memory: %s\n", buffer);

    free(buffer);
    buffer = NULL;

    printf("Memory released successfully\n");
}

void input_test()
{
    char input[MAX_SIZE];

    printf("Enter a test command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Test failed: Empty input\n");
        return;
    }

    printf("Test passed: %s\n", input);
}

int main()
{
    printf("==== Skill 22 =====\n\n");

    printf("Memory Test\n");
    memory_test();

    printf("\nInput Test\n");
    input_test();

    printf("\nAll tests completed\n");

    return 0;
}
```

```
myshell> exit
sahasrasagi@Sahasra:~/OS_Lab/skill$ nano skill_22.c
sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_22.c -o skill_22
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_22
===== Skill 22 =====

Memory Test
Allocated memory: OSSP Skill 22 Memory Test
Memory released successfully

Input Test
Enter a test command: valgrind --version
Test passed: valgrind --version

All tests completed
sahasrasagi@Sahasra:~/OS_Lab/skill$ |
```

Skill - 23:

```
sahasrasagi@Sahasra: ~/OS_L  ×  +  ∨
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20

void parse_command(char *input, char **args)
{
    int i = 0;
    char *token = strtok(input, " ");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;
}

int main()
{
    char input[500];
    char *commands[MAX_COMMANDS];
    int command_count = 0;

    printf("==== Skill 23 =====\n");
    printf("Enter a pipeline:\n");
    printf("Example: ls | grep .c | wc -l\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, "|");

    while (token != NULL && command_count < MAX_COMMANDS)
    {
        while (*token == ' ')
            token++;

        commands[command_count++] = token;
        token = strtok(NULL, "|");
    }

    if (command_count == 0)
    {
        printf("No command entered\n");
        return 1;
    }

    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    struct timeval start, end;

    gettimeofday(&start, NULL);

    for (int i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {

```



sahasrasagi@Sahasra: ~/OS_L



GNU nano 8.7.1

```
gettimeofday(&start, NULL);

for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe");
        return 1;
    }
}

for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] < 0)
    {
        perror("fork");
        return 1;
    }

    if (pids[i] == 0)
    {
        if (i > 0)
            dup2(pipes[i - 1][0], STDIN_FILENO);

        if (i < command_count - 1)
            dup2(pipes[i][1], STDOUT_FILENO);

        for (int j = 0; j < command_count - 1; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        char *args[MAX_ARGS];
        parse_command(commands[i], args);

        execvp(args[0], args);

        perror("execvp");
        exit(127);
    }
}

for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

int failed = 0;

for (int i = 0; i < command_count; i++)
{
    int status;

    waitpid(pids[i], &status, 0);

    if (WIFEXITED(status))
    {
        int code = WEXITSTATUS(status);

        if (code != 0)
        {
            printf("Job %d failed with exit code %d\n", i + 1, code);
            failed = 1;
        }
    }
}
```

```

}

int failed = 0;

for (int i = 0; i < command_count; i++)
{
    int status;

    waitpid(pids[i], &status, 0);

    if (WIFEXITED(status))
    {
        int code = WEXITSTATUS(status);

        if (code != 0)
        {
            printf("Job %d failed with exit code %d\n", i + 1, code);
            failed = 1;
        }
    }
    else if (WIFSIGNALED(status))
    {
        printf("Job %d terminated by signal %d\n", i + 1, WTERMSIG(status));
        failed = 1;
    }
}

gettimeofday(&end, NULL);

double execution_time =
    (end.tv_sec - start.tv_sec) +
    (end.tv_usec - start.tv_usec) / 1000000.0;

printf("\nPipeline processes: %d\n", command_count);
printf("Execution time: %.6f seconds\n", execution_time);

if (failed)
    printf("Pipeline status: FAILED\n");
else
    printf("Pipeline status: SUCCESS\n");

return 0;
}

```

```

sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_23.c -o skill_23
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_23
===== Skill 23 =====
Enter a pipeline:
Example: ls | grep .c | wc -l

myshell> ls
a.out      program1.c  shell_errors.log  skill_10      skill_11.c  skill_13      skill_14
output.txt program2     shellforge        skill_10.c    skill_12     skill_13.c    skill_15
program1    program2.c  shellforge.c      skill_11      skill_12.c  skill_14      skill_15

Pipeline processes: 1
Execution time: 0.005506 seconds
Pipeline status: SUCCESS
sahasrasagi@Sahasra:~/OS_Lab/skill$ |

```